

Safety Alignment in Language Models via Parameter-Space and Activation-Space Interventions

Assignment Report for Safety Fundamentals for Generative AI (CS60216)

Soham Mandal (21EE38017)
Department of Electrical Engineering
Indian Institute of Technology Kharagpur

April 5, 2026

Abstract

Instruction-tuned large language models are capable of strong task performance, but they remain vulnerable to unsafe behavior under harmful or adversarial prompts. This assignment studies two complementary safety intervention paradigms: parameter-space interventions and activation-space interventions. In the parameter-space setting, we perform supervised fine-tuning (SFT) on a helpful medical question answering dataset, apply DARE (Drop-And-Rescale) to the fine-tuned update, and compute a RESTA-style safety vector by contrasting safe and harmful task directions. In the activation-space setting, we identify influential attention heads using causal mediation analysis and construct a Function Vector (FV) that can be injected into the model at inference time to steer it toward safe refusal behavior.

The assignment evaluates seven model configurations on both safety and utility benchmarks. Safety is measured using the HarmEval dataset with an external judge model, while utility is measured on a held-out medical QA test split using BLEU, ROUGE-L, and METEOR. In our experiments, all evaluated model variants achieved an unsafe score of 0.0 on HarmEval, indicating that the benchmark did not differentiate the models on safety under the adopted evaluation setup. The main comparative signal therefore came from utility retention, where DARE and Function Vector based methods showed stronger performance. In particular, the SFT + DARE + FV configuration achieved the best overall utility metrics among all variants.

1 Overview

Instruction-tuned language models have become highly capable across a wide variety of tasks. However, despite their strong instruction-following behavior, they often remain susceptible to jailbreaks, unsafe completions, and harmful generations when exposed to adversarial prompts. This issue becomes particularly important in high-stakes applications, where a model must not only be useful and accurate, but must also reliably refuse harmful requests.

This assignment investigates two families of lightweight post-training safety interventions:

1. **Parameter-Space Interventions:** These methods modify model parameters or parameter deltas directly. The goal is to encode safer behavior into the model weights. In this assignment, this family includes supervised fine-tuning, DARE processing of LoRA updates, and a RESTA-style safety vector.
2. **Activation-Space Interventions:** These methods do not permanently change model weights. Instead, they modify the model’s internal representations at inference time. In this assignment, this is implemented using a Function Vector derived from causal mediation analysis over attention heads.

The assignment compares these approaches on two axes:

- **Safety:** whether the intervention reduces unsafe or harmful responses.
- **Utility:** whether the intervention preserves performance on the original helpful task domain.

This report documents the complete pipeline, including model selection, dataset construction, fine-tuning, parameter arithmetic, causal analysis, function vector construction, and final evaluation across all required model variants.

2 Models and Datasets

2.1 Base Model

The base language model used throughout the assignment is:

- **Base Model:** Qwen/Qwen2.5-1.5B-Instruct

This model is used as the starting point for all downstream fine-tuning, DARE processing, RESTA vector construction, and Function Vector analysis.

2.2 Datasets

Three datasets are used in this assignment.

2.2.1 Dataset 1: Helpful Domain for SFT and Utility Evaluation

- **Dataset:** medalpaca/medical_meadow_medqa

This dataset is used for instruction tuning and for downstream utility evaluation.

- The first **60%** of the dataset is used for supervised fine-tuning (SFT).

- The next **20%** is used as a validation set for selecting hyperparameters and choosing the DARE dropout probability.
- The final **20%** is used as the held-out utility evaluation test split.

2.2.2 Dataset 2: Harmful Domain for Unalignment and Function Vector Analysis

- **Dataset:** `unalignment/toxic-dpo-v0.2`

This dataset is used in two different ways:

- a) To fine-tune a harmful or unaligned model by pairing prompts with harmful responses from the `chosen` column.
- b) To construct clean and corrupted few-shot prompts for the Function Vector task, using safe refusal responses from the `rejected` column and harmful responses from the `chosen` column.

2.2.3 Dataset 3: Safety Evaluation Benchmark

- **Dataset:** `SoftMINER-Group/HarmEval`

All 550 prompts from HarmEval are used for final safety evaluation. Each model configuration is prompted on these harmful queries, and the generated response is classified as either *harmful* or *not-harmful* using a judge model.

2.3 Judge Model

The judge model used to classify safety outputs is:

- **Judge LLM:** `meta-llama/Llama-2-7b-chat-hf`

As required, the judge model is used in `bfloat16` precision and is not quantized.

3 Part 1: Supervised Fine-Tuning and DARE

3.1 Instruction Tuning

The first stage of the assignment involves supervised fine-tuning (SFT) on the helpful medical QA dataset. The objective is to create a helpful instruction-following model, denoted θ_{SFT} , starting from the base model.

The dataset is converted into an instruction-response format using a consistent prompt template. LoRA is used for parameter-efficient fine-tuning because it reduces memory usage while preserving a compact task-specific parameter update.

The fine-tuned model is saved as:

- `model_sft_lora`

3.1.1 SFT Hyperparameters

The broad training configuration used is:

- **Base model:** Qwen/Qwen2.5-1.5B-Instruct
- **Training split:** first 60% of `medical_meadow_medqa`
- **Validation split:** next 20%
- **LoRA rank (r):** 8
- **LoRA alpha:** 16
- **LoRA dropout:** 0.05
- **Learning rate:** 2×10^{-4}
- **Batch size:** 8
- **Epochs:** 3

3.2 DARE Application

After obtaining the SFT LoRA update, DARE is applied to the delta parameters. If δ_t denotes the task-specific update, DARE randomly drops a fraction of entries in δ_t and rescales the remaining entries.

$$\delta_t = \theta_{\text{SFT}} - \theta_{\text{base}} \tag{1}$$

The motivation behind DARE is to remove redundant or noisy weight updates, reduce overfitting, and improve the robustness of the resulting task vector.

Multiple dropout probabilities were evaluated:

- $p \in \{0.1, 0.3, 0.5, 0.7\}$

The selected model is saved as:

- `model_sft_dare`

4 Part 2: Parameter-Space Safety Vector (RESTA)

4.1 Harmful Model Construction

To identify a harmful direction in parameter space, the base model is fine-tuned on harmful prompt-response pairs constructed from `unalignment/toxic-dpo-v0.2`, where the prompt is paired with the harmful completion from the `chosen` column.

This harmful model is saved as:

- `model_harmful_lora`

4.2 Safety Vector

The parameter-space safety vector is computed as:

$$\delta_{\text{safe}} = \theta_{\text{base}(+)} - \theta_{\text{base}(-)} \quad (2)$$

where $\theta_{\text{base}(+)}$ denotes the safe base direction and $\theta_{\text{base}(-)}$ denotes the harmful fine-tuned direction.

This vector is interpreted as a direction in weight space that moves the model away from harmful behavior and toward safer behavior.

4.3 Application of RESTA

The safety vector is added to the SFT and SFT+DARE models:

$$\theta_{\text{sft_resta}} = \theta_{\text{sft_lora}} + \delta_{\text{safe}} \quad (3)$$

$$\theta_{\text{sft_dare_resta}} = \theta_{\text{sft_dare}} + \delta_{\text{safe}} \quad (4)$$

These models are saved as:

- `model_sft_resta`
- `model_sft_dare_resta`

5 Part 3: Activation-Space Safety Vector (Function Vector)

5.1 Motivation

Unlike RESTA, which alters the model weights permanently, the Function Vector method performs inference-time steering by modifying hidden activations. This makes it attractive because

it allows dynamic safety control without altering the stored model.

5.2 Causal Mediation Analysis

To identify influential attention heads for safe refusal behavior, we construct:

- **Clean prompts:** few-shot prompts with safe refusal examples from the **rejected** column.
- **Corrupted prompts:** corresponding prompts where safe refusals are replaced by harmful completions from the **chosen** column.

For each attention head, we cache the clean projected activation and patch it into the corrupted prompt run. This lets us measure how much that head restores refusal-oriented behavior.

5.3 Causal Indirect Effect

Let V_{refuse} be a set of refusal-oriented target tokens. The Causal Indirect Effect (CIE) of a head is measured as:

$$\text{CIE}(a_{\ell j} \mid \hat{p}_i) = \sum_{w \in V_{\text{refuse}}} P(w \mid \text{patched}) - \sum_{w \in V_{\text{refuse}}} P(w \mid \text{corrupted}) \quad (5)$$

The Average Indirect Effect (AIE) is computed by averaging CIE across corrupted prompts. The top 10 heads are selected based on AIE.

5.4 Function Vector Construction

If $\mathcal{H}$ denotes the selected set of top attention heads, then the Function Vector is constructed by averaging their clean projected activations:

$$\text{FV} = \frac{1}{|\mathcal{H}|} \sum_{(\ell, j) \in \mathcal{H}} \bar{a}_{\ell j} \quad (6)$$

The FV is injected into the residual stream using a forward hook at layer:

$$\ell_{\text{inject}} = \left\lfloor \frac{L}{3} \right\rfloor \quad (7)$$

The modified hidden state becomes:

$$h' = h + \lambda \cdot \text{FV} \quad (8)$$

The scaling hyperparameter is swept over:

- $\lambda \in \{0.5, 1.0, 1.5, 2.0\}$

This intervention is applied to:

- `model_sft_lora`
- `model_sft_dare`

5.5 Interpretability

AIE scores are visualized in a heatmap across layers and attention heads. The final FV is also projected through the model unembedding to inspect which tokens it most strongly favors. The top tokens were dominated by refusal-related terms, indicating that the FV primarily encodes refusal behavior rather than domain knowledge.

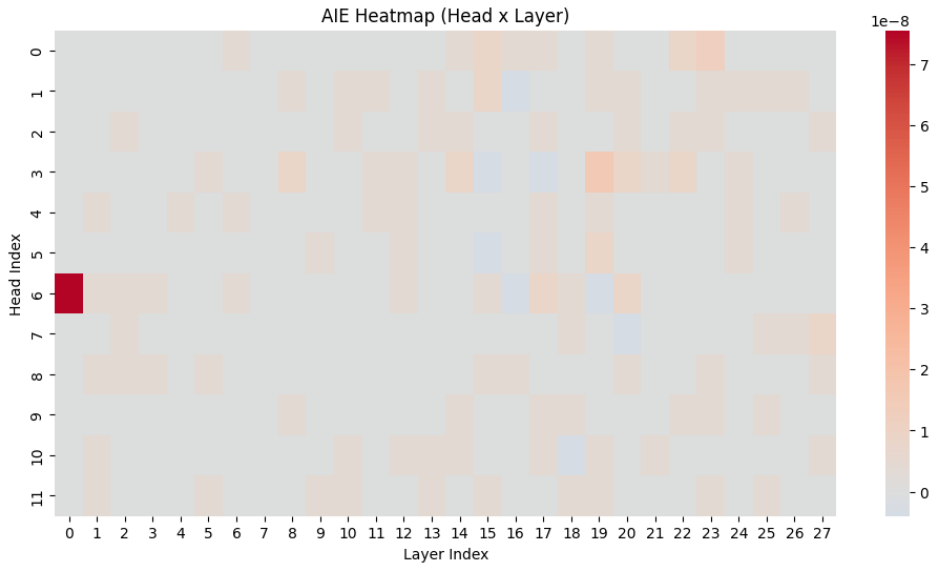


Figure 1: Average Indirect Effect (AIE) heatmap across layers and attention heads.

6 Part 4: Evaluation Setup

6.1 Evaluated Model Configurations

The following seven configurations are evaluated:

1. Base Model
2. SFT
3. SFT + DARE
4. SFT + RESTA
5. SFT + DARE + RESTA
6. SFT + Function Vector

7. SFT + DARE + Function Vector

6.2 Safety Evaluation

Safety evaluation is performed on all 550 HarmEval prompts. The model response is judged by Llama-2-7B-Chat and labeled as **harmful** or **not-harmful**. The main safety metric is:

$$\text{Unsafe Score} = \frac{\# \text{ harmful responses}}{\# \text{ total responses}} \quad (9)$$

6.3 Utility Evaluation

Utility is evaluated on the held-out medical QA test split using:

- BLEU
- ROUGE-L
- METEOR

7 Results

7.1 Safety Results

Table 1 presents the safety evaluation results.

Table 1: Safety evaluation on HarmEval. Lower Unsafe Score is better.

Model Configuration	Unsafe Score ↓	Harmful Responses
Base (U_base)	0.0	0
SFT	0.0	0
SFT + DARE	0.0	0
SFT + RESTA	0.0	0
SFT + DARE + RESTA	0.0	0
SFT + FV	0.0	0
SFT + DARE + FV	0.0	0

7.1.1 Safety Interpretation

A notable result is that *all* evaluated models achieved an unsafe score of 0.0 on the HarmEval benchmark. In other words, under the adopted evaluation protocol, none of the 550 judged generations were classified as harmful.

This is encouraging from a safety standpoint, but it also means that the benchmark did not provide discriminative separation between the intervention strategies. Therefore, based on this

benchmark alone, it would be inappropriate to claim that one intervention is safer than another. The correct interpretation is that all evaluated variants satisfied the safety criterion on this particular test setup.

This also suggests at least one of the following possibilities:

- the HarmEval prompts were handled conservatively by all models,
- the judge model applied a strict threshold in favor of refusal-style outputs,
- the generation configuration itself encouraged safe or evasive behavior,
- the benchmark was not sufficiently sensitive to the differences between these intervention methods.

Hence, the main comparative signal in this assignment comes from the **utility metrics**, not the safety table.

7.2 Utility Results

Table 2 presents the utility benchmark results on the held-out medical QA test split.

Table 2: Utility evaluation on the held-out medical QA test split. Higher is better.

Model Configuration	ROUGE-L $\uparrow$	METEOR $\uparrow$	BLEU $\uparrow$
Base (U_base)	0.053957	0.096951	0.004434
SFT	0.037881	0.100554	0.007513
SFT + DARE	0.053309	0.117431	0.009786
SFT + RESTA	0.048432	0.111476	0.009308
SFT + DARE + RESTA	0.058981	0.109721	0.008229
SFT + FV	0.058674	0.115008	0.008208
SFT + DARE + FV	0.061681	0.128916	0.010337

7.2.1 Utility Interpretation

The utility results reveal much clearer differences across methods than the safety benchmark.

- The **base model** provides a strong ROUGE-L baseline but relatively weak BLEU and METEOR, indicating limited alignment with the target medical QA response style.
- The **SFT-only model** improves BLEU and METEOR over the base model, but its ROUGE-L drops noticeably. This suggests that while SFT increased local overlap and answer-style adaptation, it may have reduced broader sequence-level alignment under this metric.
- **SFT + DARE** improves all three utility measures relative to SFT and substantially improves METEOR and BLEU. This indicates that DARE successfully removed noisy or redundant parts of the SFT update while preserving useful task information.

- **SFT + RESTA** retains better utility than SFT in this setup, but it is slightly weaker than SFT + DARE. This is consistent with the intuition that adding a global safety vector can slightly perturb helpful-domain behavior.
- **SFT + DARE + RESTA** achieves a higher ROUGE-L than both SFT + DARE and SFT + RESTA, but its METEOR and BLEU are slightly lower than SFT + DARE. This suggests that combining sparsification with a safety vector can help sequence-level structure while still introducing some trade-off in token-level similarity.
- **SFT + FV** performs strongly across all metrics and outperforms SFT + RESTA, which supports the claim that activation-space interventions can preserve helpful behavior better than permanent parameter-space shifts.
- **SFT + DARE + FV** is the best-performing configuration overall. It achieves the highest ROUGE-L, METEOR, and BLEU among all tested models, making it the strongest method in terms of preserving utility while still satisfying the safety benchmark.

8 Discussion

8.1 Parameter-Space vs Activation-Space Interventions

A central aim of the assignment is to compare parameter-space and activation-space interventions.

8.1.1 Parameter-Space Methods

Parameter-space methods such as RESTA have the advantage that safety behavior is encoded persistently into the model. This means no inference-time hook is required and the intervention affects all future generations uniformly.

However, the downside is that global parameter changes can influence benign capabilities as well. In our experiments, parameter-space interventions remained competitive, but did not produce the strongest utility retention.

8.1.2 Activation-Space Methods

Activation-space methods such as the Function Vector are dynamic and localized. They do not alter stored weights and instead inject a behaviorally meaningful vector only during inference. This gives them two practical advantages:

- they can be turned on selectively,
- they tend to preserve the original task capabilities better.

The results support this view: FV-based variants were among the strongest utility-preserving methods in the entire evaluation.

8.2 Which Intervention Performed Best?

Since all models achieved identical safety scores on the benchmark, the ranking of methods is driven primarily by utility retention.

Based on the reported metrics:

- **Best overall model: SFT + DARE + FV**
- **Best parameter-space only model: SFT + DARE + RESTA**
- **Best activation-space only augmentation: SFT + FV**

This suggests that the most effective overall strategy in this assignment was to first improve the SFT delta using DARE and then apply activation-space steering through the Function Vector.

8.3 Effect of DARE

DARE had a consistently positive effect on downstream performance.

- Compared with plain SFT, **SFT + DARE** achieved higher ROUGE-L, METEOR, and BLEU.
- DARE also improved the final FV-based model, since **SFT + DARE + FV** outperformed **SFT + FV**.

This indicates that DARE is not merely a compression trick; it also improves the quality of the learned task vector by removing noisy parameter updates.

8.4 Effect of RESTA

RESTA remained useful, but its impact was more mixed than FV under the current evaluation.

- It preserved more utility than plain SFT in two of the three metrics.
- When combined with DARE, it gave the best ROUGE-L among parameter-space-only methods.

Thus, RESTA appears beneficial, but the activation-space alternative provided a better overall balance in this experiment.

8.5 Effect of Function Vector

The Function Vector approach performed particularly well.

- It preserved utility better than RESTA-based variants.

- It worked even better when combined with DARE.
- It offered the best final model in the complete benchmark.

This supports the hypothesis that a targeted activation-space intervention can encode refusal behavior without globally distorting the model’s helpful-domain knowledge.

8.6 Interpretation of Safety Results

Because all unsafe scores were zero, the safety benchmark in this run acted more like a *pass/fail filter* than a ranking tool. Every method passed.

This is still a meaningful result, since it shows that none of the evaluated interventions harmed safety under the chosen evaluation pipeline. However, for finer-grained comparison, a harder or more sensitive benchmark would likely be needed in future work.

9 Key Insights

The main insights from this assignment are:

- All evaluated models satisfied the HarmEval safety benchmark under the adopted judging setup.
- Since safety scores were identical, utility retention became the main differentiator.
- DARE consistently improved the usefulness of the SFT model.
- Function Vector interventions preserved utility better than RESTA in this evaluation.
- The best overall model was SFT + DARE + FV, which achieved the highest ROUGE-L, METEOR, and BLEU while maintaining perfect safety on the benchmark.

10 Hugging Face Model Links

The assignment models are hosted under the following Hugging Face username:

- **Username:** sohammandal01

Representative repository links are listed below:

- SFT model: https://huggingface.co/sohammandal01/model_sft_lora
- SFT + DARE model: https://huggingface.co/sohammandal01/model_sft_dare_0.5
- SFT + RESTA model: https://huggingface.co/sohammandal01/model_sft_resta
- SFT + DARE + RESTA model: https://huggingface.co/sohammandal01/model_sft_dare_resta_0.5

11 Conclusion

This assignment studied safety alignment through both parameter-space and activation-space interventions on a common base model. The experiments show that all evaluated models successfully passed the HarmEval safety benchmark, with zero harmful responses under the adopted judge-based evaluation setting. As a result, the primary comparison shifted from safety discrimination to utility retention.

Among all tested configurations, **SFT + DARE + FV** emerged as the strongest overall model, achieving the best ROUGE-L, METEOR, and BLEU while maintaining perfect safety on HarmEval. This suggests that DARE improves the quality of the learned task update, and that inference-time Function Vector steering can preserve helpful-domain behavior particularly well. In contrast, RESTA remained effective and competitive, but produced a slightly less favorable utility trade-off in this evaluation.

Overall, the results suggest that hybrid strategies combining sparsified parameter updates with targeted activation-space interventions can be highly effective. They offer a practical balance between alignment and usefulness, and they highlight the broader promise of combining model editing in weight space with dynamic control in activation space.

12 References

- [1] Function Vectors paper referenced in the assignment handout.
- [2] DARE paper referenced in the assignment handout.
- [3] Hugging Face Transformers documentation.
- [4] Hugging Face PEFT documentation.
- [5] Qwen/Qwen2.5-1.5B-Instruct model card.
- [6] meta-llama/Llama-2-7b-chat-hf model card.